

## IIC 2413 – Bases de Datos

### Tarea 2 — Manejo de un Workload en un entorno realista

**Grupos:** 3 personas  
**Fecha de entrega:** Viernes 29 de Mayo, a las 20:00 hrs.  
**Modalidad:** Informe escrito (PDF) + archivos SQL

## 1. Contexto

Son los administradores de una librería online, actualmente con muchísimos problemas de rendimiento, los clientes se quejan por lo lenta que funciona su página. Recibirán los datos de la librería online, y una carga de trabajo típica para la empresa (**workload.sql**) con **100 consultas** que la aplicación ejecuta. En la base de datos de la librería, todas las tablas tienen un B+ tree en la llave primaria.

**Su objetivo es uno solo:** hacer que esa carga corra más rápido. Tienen dos herramientas para lograrlo:

- Crear hasta **4 índices** (**CREATE INDEX**).
- Crear hasta **1 vista materializada** (**CREATE MATERIALIZED VIEW**).

No pueden crear más índices o vistas materializadas, y no pueden interferir con otras áreas de la base de datos: no modifiquen `postgresql.conf`, no cambien `work_mem` ni `shared_buffers`, no creen tablas auxiliares, no reescriban consultas que no consuman su vista materializada.

## 2. Detalle de los recursos recibidos

Cada grupo recibe un `.zip` con su `group_id`:

```
group<N>/
├── schema.sql ..... Esquema
├── data/ ..... CSVs con datos de su grupo
├── load.sql ..... para cargar los datos
├── workload.sql ..... Las 100 consultas a optimizar
└── run_workload.py ..... Mide tiempos
```

**Cargar la base:**

```
cd path/to/group<N>/
createdb bookstore_g<N>
psql -d bookstore_g<N> -f schema.sql
psql -d bookstore_g<N> -f load.sql
```

La carga toma unos minutos. La base puede llegar a usar hasta 3GB en disco.

## 3. Pasos a seguir

### Paso 1 — Medir la línea base y clasificar consultas

Este es el punto más crítico de su tarea, pues les va a sentar las bases para tomar las decisiones futuras. Primero, corran el workload **antes** de comenzar (fijense que su base de datos debe tener el nombre correcto):

```
python3 run_workload.py --db bookstore_g<N> --csv times_baseline.csv
```

El script imprime el **tiempo total de la carga** (este es el número que usamos para evaluar) y escribe un CSV con el tiempo de cada consulta individualmente.

**Importante:** Para normalizar los efectos del caché en la base de datos, se debe correr el workload al menos dos veces, reportando la última corrida (esto se conoce como experimentos en caliente).

Ahora, lo que normalmente se hace es clasificar las consultas. De las 100 consultas del workload, ¿hay consultas similares? ¿Qué consultas usan las mismas tablas y cómo las están usando? ¿Qué tipo de consultas son las más importantes a la hora de optimizar? Documenten su clasificación en el reporte; les va a servir para Pasos 2 y 3.

## Paso 2 — Decidir dónde poner los índices

Proponer **hasta 4 índices**. Para cada uno escriban:

1. El `CREATE INDEX` (en `indexes.sql`).
2. Por qué este índice es importante.
3. Cuánto impacta el tiempo de ejecución antes vs. después del índice (sobre una o más consultas representativas)

## Paso 3 — Materializar en caché

Algunas consultas no se aceleran con índices: el costo no está en el **acceso** a los datos, sino en el **cómputo** (joins muy grandes, agregaciones que necesitan ver todas las filas, semi-joins sobre tablas grandes, etc.). Para esas conviene **precalcular** el resultado y dejarlo guardado en disco, de modo que las consultas siguientes lo **lean** en vez de recalcularlo.

En PostgreSQL esto se hace con una **vista materializada**:

```
CREATE MATERIALIZED VIEW <nombre> AS
<consulta SELECT que produce el resultado a precalcular>;
```

Postgres ejecuta la consulta una vez y guarda el resultado en disco como si fuera una tabla. Pueden incluso ponerle `CREATE INDEX` encima como a cualquier tabla. Para recalcularla cuando los datos subyacentes cambian se usa `REFRESH MATERIALIZED VIEW <nombre>;` entre refrescos los datos quedan “congelados” — ese es el costo de mantención. En su caso, no aplica este refresco, pues los datos están estáticos.

**Importante:** crear la matview **no** hace que las consultas existentes la usen automáticamente. Hay que **reescribir** la consulta original para que la consuma.

**Ejemplo.** Supongamos que su aplicación ejecuta:

```
SELECT COUNT(*) FROM (
    SELECT author_id FROM books
    GROUP BY author_id HAVING COUNT(*) > 50
) sub;
```

y ven en el plan que esta consulta no anda muy bien. La materialización:

```
CREATE MATERIALIZED VIEW author_book_counts AS
SELECT author_id, COUNT(*) AS n_books FROM books GROUP BY author_id;
```

crea una tabla pequeña con la cuenta por autor, que es muy útil para la consulta de arriba. Pero la consulta de la aplicación **sigue tocando books directamente**; para que use la matview hay que reescribirla:

```
SELECT COUNT(*) FROM author_book_counts WHERE n_books > 50;
```

Las dos consultas devuelven **el mismo número**, pero la segunda es trivial.

**Reglas para esta tarea:**

- Tienen 1 sola **matview** disponible. Eljanla bien.
- Pueden **reescribir** las consultas del workload que consuman su matview, **siempre que la reescritura retorne exactamente el mismo resultado** para los mismos parámetros (mismas filas, misma cantidad). Si lo dudan, pruébenlo: corran la original y la reescrita y comparen.
- Las reescrituras se entregan en un archivo **workload\_after.sql** (copia de **workload.sql** con los cambios). El *after* se mide contra ese archivo; el *baseline* se mide contra el original.
- No reescriban consultas que no usan la matview.

En **matview.sql** entreguen el **CREATE MATERIALIZED VIEW** (y opcionalmente un **CREATE INDEX** sobre ella). En el reporte expliquen qué clase de consulta atacaron, por qué los índices no eran suficientes, qué reescritura(s) hicieron, y qué costo de mantención asume su solución.

## Paso 4 — Re-medir y reportar

Apliquen sus **indexes.sql** y **matview.sql** sobre una base recién cargada (importante: **no** sobre la misma instancia donde experimentaron, para evitar contaminar las estadísticas), y vuelvan a medir:

```
psql -d bookstore_g<N> -f indexes.sql
psql -d bookstore_g<N> -f matview.sql
psql -d bookstore_g<N> -c "ANALYZE;"
```

```
# Si reescribieron alguna consulta para que use la matview, midan
# contra workload_after.sql (su copia de workload.sql con los cambios).
# Si no reescribieron nada, usen workload.sql directamente.
python3 run_workload.py --db bookstore_g<N> \
    --workload workload_after.sql \
    --csv times_after.csv
```

Anoten el nuevo tiempo total y completen el reporte (siguiente sección).

## 4. Reporte

Un único PDF de **máximo 6 páginas** reportando su trabajo, estructurado de acuerdo a los 4 pasos de arriba:

1. Paso 1 - Línea base y análisis de workload
2. Paso 2 - Índices
3. Paso 3 - Materialized view
4. Paso 4 - Mediciones finales

## 5. Evaluación

| Componente                | %  | Qué estamos midiendo                                                     |
|---------------------------|----|--------------------------------------------------------------------------|
| Speedup objetivo          | 50 | Cuán cerca quedó su tiempo total al óptimo.                              |
| Calidad de las decisiones | 30 | Justificación de cada índice y de la matview.                            |
| Reporte                   | 20 | Claridad, evidencia (planes y tablas), honestidad sobre lo que probaron. |

**Sobre el speedup objetivo.** Para cada grupo hemos calculado un speedup optimo basado en calculos numéricos.

La nota de la componente depende de cuánto logran acelerar. Depende del óptimo, y si es necesario dependerá a su vez de la distribución de los speedups de cada grupo.

## 6. Entrega

Un .zip llamado `group<N>_submission.zip` con:

```
group<N>_submission/
├── reporte.pdf.....El PDF descrito arriba
├── indexes.sql.....Sus CREATE INDEX (máximo 4)
├── matview.sql.....Su CREATE MATERIALIZED VIEW (máximo 1)
├── workload_after.sql.....Su workload con las reescrituras (si las hubo); si no, copia de workload.sql
├── times_baseline.csv.....Tiempo por consulta en la línea base
├── times_after.csv.....Tiempo por consulta después de sus cambios
└── README.md ..... Nombres del grupo, versión de Postgres, comandos para reproducir
```

## 7. Política de uso de LLMs

Se mantiene la política de la Tarea 1.